

Lekce 7: Práce s webovými prvky a zpracování uživatelských vstupů

1. Osnova obsahu

A. Úvod

- **Proč je interakce s webovými prvky důležitá:**
 - Ověřuje, jak uživatelé pracují s prvky, jako jsou tlačítka, vstupní pole, zaškrťovací políčka, rozbalovací nabídky atd.
 - Zajišťuje, že webové aplikace reagují na uživatelské vstupy dle očekávání.

Proč je interakce s webovými prvky důležitá

Vysvětlení:

- **Simulace reálného světa:**

Interakce s webovými prvky simuluje chování reálných uživatelů v aplikaci. Automatizované testy napodobující kliknutí, psaní a další akce pomáhají zajistit, že uživatelské rozhraní reaguje správně v každé situaci.
- **Validace chování UI:**

Potvrzuje, že uživatelské akce (např. odeslání formuláře, výběr možnosti nebo vyvolání modálního okna) vedou k očekávaným změnám v DOM, jako je zobrazení chybových hlášek, aktualizace obsahu nebo přechod na novou stránku.
- **Odhalení regresních chyb:**

Automatizované interakce mohou rychle odhalit regresní chyby v UI. Pokud vývojář udělá změnu ovlivňující funkčnost tlačítka, testy interakce na to ihned upozorní.
- **End-to-End testování:**

Testování interakcí je klíčové pro end-to-end (E2E) testy, kde je ověřována celá uživatelská cesta – od načtení stránky přes sekvenci akcí až po sledování výsledků.

B. Výběr prvků v Cypress

- **CSS selektory (základní metoda):**
 - Základní selektory: `.class`, `#id`, `tag`.
 - Složené selektory: `div > button`, `input[type="text"]`.
- **Použití atributů `data-*`:**
 - Nejlepší praxe pro stabilní selektory v automatizaci:

```
cy.get('[data-testid="login-button"]');
```

- **Metody procházení:**

- Nalezení podřízených prvků: `cy.get('div').find('button')` .
- Navigace ve struktuře DOM: `.parent()` , `.children()` , `.next()` , `.prev()` .

- **Nejlepší postupy:**

- Pro stabilitu testů používejte atributy `data-*` .
- Vyhýbejte se křehkým selektorům, jako je `nth-child` .

cy.get() vs cy.find()

Klíčový rozdíl:

Použijte `cy.get()` pro dotaz na celý globální DOM. Použijte `cy.find()` , pokud potřebujete zúžit vyhledávání pouze na prvky uvnitř konkrétního kontejneru či kontextu.

cy.get():

- **Použití:**

`cy.get()` vybírá prvky z celého DOM na základě zadaného selektoru.

- **Příklad:**

```
// Vybere prvek s atributem data-testid z celé stránky
cy.get('[data-testid="login-form"]');
```

cy.find():

- **Použití:**

`cy.find()` hledá potomky uvnitř dříve vybraného prvku.

- **Příklad:**

```
// Nejprve vyberte rodičovský prvek
cy.get('[data-testid="login-form"]')
// Poté najděte vstupní pole uvnitř tohoto formuláře
.find('[data-testid="username-input"]');
```

Proč je atribut `data-testid` lepší než ID či selektor třídy

Vysvětlení:

- **Stabilita při změnách UI:**

Atributy `data-testid` jsou určeny výhradně pro testování. Na rozdíl od ID nebo tříd nebývají ovlivněny změnou návrhu nebo refaktoringem.

- **Oddělení odpovědnosti:**

Oddělení testovacích selektorů od stylovacích nebo strukturálních selektorů zaručuje, že testy

zůstávají odolné vůči vizuálním změnám.

- **Popisné a srozumitelné:**

Jasně ukazují, že atribut je určen pro testování (např. `data-testid="login-button"`), což usnadňuje správu a pochopení testů.

- **Zabránění konfliktům:**

ID a třídy se používají pro styly a rozvržení, což může vést ke konfliktům nebo neočekávanému chování, pokud se stejné selektory použijí i pro testy.

Příklad:

```
<!-- Použití data-testid pro testování -->
<button data-testid="submit-button">Odeslat</button>
```

```
// Cypress test používající data-testid
cy.get('[data-testid="submit-button"]').click();
```

C. Akce nad webovými prvky

1. Klikání na prvky:

- Použijte `cy.click()` k simulaci kliknutí.
- Příklad:

```
cy.get('[data-testid="submit-button"]').click();
```

2. Psaní do vstupních polí:

- Použijte `cy.type()` k simulaci psaní.
- Příklad:

```
cy.get('[data-testid="username-input"]').type('testUser');
```

- **Simulace klávesových událostí:**

- Použijte `.type()` se speciálními klávesami jako `{enter}`, `{backspace}`.
- Příklad:

```
cy.get('[data-testid="search-input"]').type('Cypress{enter}');
```

3. Mazání vstupního pole:

- Použijte `.clear()` k vymazání obsahu pole.

- Příklad:

```
cy.get('[data-testid="username-input"]').clear();
```

4. Volba možností v rozbalovacích polích:

- Použijte `.select()` pro `<select>` prvky.
- Příklad:

```
cy.get('[data-testid="dropdown"]').select('Option 1');
```

5. Zaškrťovací políčka a přepínače:

- `.check()` pro zaškrtnutí, `.uncheck()` pro odškrtnutí checkboxů.
- Příklad:

```
cy.get('[data-testid="accept-terms"]').check();
```

6. Odesílání formulářů:

- Použijte `.submit()` pro simulaci odeslání formuláře.
- Příklad:

```
cy.get('[data-testid="login-form"]').submit();
```

7. Najetí kurzorem na prvek:

- Cypress nemá vlastní akci `hover`; použijte `.trigger('mouseover')`.
- Příklad:

```
cy.get('[data-testid="menu-item"]').trigger('mouseover');
```

D. Validace interakcí

- **Asertace (ověření):**

- Potvrďte, že akce vedou k očekávanému stavu nebo chování.
- Příklad:

```
cy.get('[data-testid="success-message"]')
  .should('be.visible')
  .and('contain', 'Login Successful!');
```

- **Řetězení příkazů:**

- Kombinujte akce a asertace pro jasné a stručné testy.

- Příklad:

```
cy.get('[data-testid="submit-button"]')
  .click()
  .get('[data-testid="error-message"]')
  .should('be.visible')
  .and('contain', 'Invalid credentials');
```

E. Pokročilé zpracování vstupů

- **Nahrávání souborů:**

- Použijte `cy.selectFile()` pro vstupní prvky typu soubor.
- Příklad:

```
cy.get('[data-testid="file-upload"]').selectFile('cypress/fixtures/sample.pdf');
```

- **Interakce s deaktivovanými prvky:**

- Ověření nebo vyvolání událostí na deaktivovaných prvcích:

```
cy.get('[data-testid="submit-button"]').should('be.disabled');
```

`cy.trigger()`

Vysvětlení:

- **Účel:**

`cy.trigger()` se používá k simulaci událostí, které nemají vlastní Cypress příkaz (např. `mouseover`, `keydown` atd.). To je užitečné pro testování chování prvků na vlastní nebo složité uživatelské interakce.

- **Příklad:**

```
// Vyvolání události mouseover na prvku
cy.get('[data-testid="menu-item"]').trigger('mouseover');
```

- **Kdy použít:**

Použijte `cy.trigger()`, pokud potřebujete simulovat události nad rámec standardních akcí kliknutí nebo psaní – zejména pro dynamické UI změny jako tooltipy nebo rozbalovací nabídky zobrazované při hoveru.

cy.within()

Vysvětlení:

- **Účel:**

cy.within() omezuje následující Cypress příkazy na konkrétní prvek. Hodí se, pokud chcete omezit dotazy na konkrétní kontejner, čímž zajistíte, že selektory budou vyhledávat pouze v jeho rozsahu.

- **Příklad:**

```
// Omezte všechny následující příkazy na element formuláře
cy.get('[data-testid="login-form"]').within(() => {
  cy.get('[data-testid="username-input"]').type('testUser');
  cy.get('[data-testid="password-input"]').type('password123');
});
```

- **Výhody:**

- Zvyšuje spolehlivost testů tím, že brání falešně pozitivním výsledkům z podobných prvků jinde na stránce.
- Zjednodušuje selektory tím, že zužuje vyhledávací kontext.

Proč použít plugin Cypress Real Events (cypress-real-events) místo nativních Cypress událostí

Vysvětlení:

- **Nativní omezení:**

Vestavěné Cypress příkazy (jako cy.click() , cy.type()) dobře fungují pro mnoho interakcí, ale někdy plně nesimulují skutečné uživatelské chování. Například složitější pohyby myši nebo komplexnější sekvence událostí nemusí být vyvolány stejně jako v reálném prohlížeči.

- **Cypress Real Events plugin:**

- **Účel:**

Plugin cypress-real-events posílá skutečné prohlížečové události (např. reálné pohyby myši nebo klávesové události), což je věrnější skutečnému uživatelskému chování.

- **Výhody:**

- **Přesnější simulace:** Lépe replikuje nativní chování prohlížeče, hlavně pro vícestupňové interakce.
- **Vyšší spolehlivost testů:** Pomáhá v situacích, kdy nativní Cypress události nevyvolají obslužné programy, např. složitější drag-and-drop nebo hover efekty.

- **Příklad:**

```
// Nejprve nainstalujte plugin:  
// npm install cypress-real-events --save-dev  
  
// Importujte plugin ve vašem Cypress support souboru (např. cypress/support/commands.js)  
import 'cypress-real-events/support';  
  
// Poté v testu použijte:  
cy.get('[data-testid="drag-item"]').realMouseDown();  
cy.get('[data-testid="drop-zone"]').realMouseUp();
```

- **Kdy použít:**

Plugin použijte tam, kde je potřeba simulovat sekvenci událostí co nejvěrněji – například drag-and-drop, hover efekty s prodlevou nebo složité scénáře pohybu myši.

2. Praktické aktivity

A. Cvičení 1: Odeslání formuláře

- **Cíl:**
 - Vytvořte přihlašovací formulář s atributy `data-testid`.
 - Simulujte vstupy uživatele (uživatelské jméno, heslo), kliknutí na tlačítko přihlášení a ověřte úspěšné či chybové zprávy.
- **Návrh webové funkcionality:**
 - Jednoduchý přihlašovací formulář s validací.

B. Cvičení 2: Práce s rozbalovacími poli a zaškrťovacími políčky

- **Cíl:**
 - Přidejte rozbalovací nabídku pro výběr role uživatele a zaškrťovací políčko pro souhlas s podmínkami.
 - Ověřte, že formulář lze odeslat pouze při vyplnění všech polí a označení checkboxu.
- **Návrh webové funkcionality:**
 - Formulář zabraňující odeslání bez splnění validačních kritérií.

C. Cvičení 3: Simulace klávesových událostí

- **Cíl:**
 - Vytvořte vyhledávací pole, které bude filtrovat výsledky dynamicky podle zadávaného textu.
 - Otestujte funkčnost vyhledávání pomocí `.type()` a `.clear()`.
- **Návrh webové funkcionality:**
 - Vyhledávací pole zobrazující výsledky v reálném čase.

3. Možné dotazy studentů

1. **Proč mám používat atributy `data-*` místo CSS selektorů?**
 - **Odpověď:** CSS selektory se mohou měnit při úpravách designu, kdežto atributy `data-*` jsou stabilní a slouží vývojářům/testování, což zajišťuje spolehlivost testovacích skriptů.
2. **Jak mohu v Cypressu simulovat najetí myši na prvek?**
 - **Odpověď:** Použijte `.trigger('mouseover')`, protože Cypress nemá speciální metodu pro hover.
3. **Mohu pracovat s prvky, které jsou skryté nebo deaktivované?**
 - **Odpověď:** Cypress standardně zakazuje interakci se skrytými nebo deaktivovanými prvky, ale můžete použít `.invoke('show')` nebo změnit atributy pro simulaci interakcí.
4. **Jak ověřím, že formulář funguje správně?**
 - **Odpověď:** Kombinujte akce (např. psaní, klikání) s asercemi pro ověření očekávaných výsledků, jako je zobrazení úspěšných/chybových zpráv.

4. Doplnkové materiály

- **Oficiální dokumentace:**
 - [Cypress Commands](#)
 - [Cypress Best Practices](#)
- **Videa a tutoriály:**
 - [Traversy Media: Cypress Crash Course](#)
 - [The Net Ninja: Cypress Testing Tutorials](#)
- **Interaktivní nástroje:**
 - Procvičujte například s [TodoMVC](#).
- **Weby pro trénink automatizace testování**

- [Advantage eshop demo](#)
- [DemoBlaze eshop](#)
- [Tools QA](#)
- [UI Test Automation Playground](#)
- [Sauce Labs](#)
- [Cypress Playground](#)
- [Automation Exercise](#)
- [Practice Test Automation Website for Web UI & API](#)

Navržené rozdělení lekce na 3 hodiny

1. hodina: Základy výběru a interakce s prvky (60 minut)

- Úvod do selektorů prvků.
- Provádění základních akcí jako klikání a psaní.
- Praktická aktivita: Vyplnění a odeslání formuláře.

2. hodina: Pokročilé interakce a zpracování vstupů (60 minut)

- Práce s rozbalovacími poli, zaškrťovacími políčky a nahráváním souborů.
- Simulace klávesových událostí a práce s deaktivovanými prvky.
- Praktická aktivita: Dynamická práce s rozbalovací nabídkou a vyhledávacím polem.

3. hodina: Validace uživatelských akcí (60 minut)

- Psaní asercí pro různé scénáře.
- Kombinování akcí a asercí v testovacích případech.
- Praktická aktivita: Testování a validace chování formuláře podle vstupu.